

Multi-Level AI Marketing Agency

A Flask-based AI marketing agent that replaces a traditional agency workflow. Upload a product image, describe it, set a tone -- and get back audience personas, ad copy variants, CTA scores, and compliance flags. Then refine the output interactively through a built-in assistant panel with sliders and a chat interface.

Built as a local-first POC: no cloud databases, no paid inference for text generation.

What it does

Given a product image + description + tone, the pipeline produces:

- Audience persona -- age range, interests, platform behavior, persona label
- Two ad copy variants -- persona-aware, tone-matched, ~150 chars + CTA
- CTA analysis -- scores each variant's CTA and suggests a stronger alternative
- Compliance flags -- regex-based guardrails (no LLM, fast + deterministic)
- Vision analysis -- product type, tags, buyer signals, mood, hashtags

After generation, the assistant agent lets you refine the campaign interactively:

- Sliders for Tone, Audience Age, and Formality -- instantly re-run only the affected agents
- Chat interface -- type freeform feedback ("make the CTA more urgent", "target eco-conscious buyers") and the assistant reasons about which pipeline stages to re-run
- Proactive suggestions -- the assistant automatically analyses the output on page load and surfaces 2-3 specific improvements
- Routing display -- shows exactly which agents ran (e.g. Planner -> Copywriter -> CTA -> Output)

Optionally, upload brand briefs, product specs, or competitor docs to a local knowledge base -- retrieved chunks are injected into the audience, copywriter, and CTA stages automatically.

Agent pipeline

```
color.extract()           LOCAL -- Pillow / ColorThief
vision.analyze()          OpenRouter (nvidia/nemotron-nano-12b-v2-vl:free, fallback: meta-llama/llama-3.1-vision-instruct)
rag.retrieve()            LOCAL -- ChromaDB + nomic-embed-text via Ollama
audience.segment()       Ollama -- mistral:7b
copywriter.generate()     Ollama -- gemma4:e2b
guardrails.check()        LOCAL -- pure Python regex
cta_optimizer.optimize()  Ollama -- mistral:7b

assistant.suggest()       Ollama -- mistral:7b (proactive suggestions, fires on page load)
assistant.get_routing()   Ollama -- mistral:7b (ReAct routing for /refine requests)
```

See ARCHITECTURE.md for the full pipeline diagram.

Model split

Two Ollama models are used, matched to task type:

Model	Used for	Why

gemma4:e2b	Ad copywriter	Creative prose -- quality matters most here

mistral:7b | Audience, CTA, assistant | Structured JSON tasks -- fast and reliable

Setup

Prerequisites

- Python 3.10+
- Ollama running locally with the following models pulled:

```
ollama pull gemma4:e2b
ollama pull mistral:7b
ollama pull nomic-embed-text
```

- An OpenRouter API key (free tier works)

Install

```
git clone https://github.com/YashAlwani/Multi-Level-AI-marketing-agency.git
cd Multi-Level-AI-marketing-agency
pip install -r requirements.txt
```

Configure

```
cp config.example.py config.py
```

Edit config.py and set your OPENROUTER_API_KEY. Everything else works out of the box.

Run

```
python app.py
```

Open <http://localhost:5000>.

Knowledge base (RAG)

Upload PDF, TXT, or MD files from the home page or at /docs. Documents are chunked, embedded locally via nomic-embed-text, and stored in ChromaDB. On each campaign run, the top 4 most relevant chunks are retrieved and injected into the audience, copywriter, and CTA prompts.

- Duplicate uploads are safe -- content is hashed and skipped if already indexed
- The knowledge base persists in chroma_store/ (local, git-ignored)

File layout

app.py	Flask routes + pipeline orchestration
config.py	Secrets + runtime config (git-ignored)
config.example.py	Template -- copy to config.py
agents/	
vision.py	OpenRouter vision analysis (primary + fallback model)
color.py	Local color palette extraction
audience.py	Ollama audience segmentation (mistral:7b)
copywriter.py	Ollama ad copy generation (gemma4:e2b)
guardrails.py	Regex compliance checks
cta_optimizer.py	Ollama CTA scoring + suggestions (mistral:7b)

rag.py	ChromaDB ingestion + retrieval
assistant.py	ReAct orchestrator -- routing, suggestions, refinement (mistral:7b)
templates/	
index.html	Upload form + knowledge base upload
result.html	Campaign report + assistant refinement panel
docs.html	Knowledge base document viewer
docs/	Plain-English design documents (see below)
diagrams/	Mermaid sources + rendered PNGs (00-INDEX.txt explains each)
decision_logs/	The eight decision logs, DL-SUMMARY.txt, EVIDENCE-MANIFEST.md
ARCHITECTURE.md	ASCII pipeline diagram (technical source)

Documentation

The design is explained in plain language, without reading code, in four design documents under docs/:

- gap-analysis.txt -- before vs after: how a human agency makes a campaign, and how the agent pipeline changes it
- pipeline-architecture.txt -- how one campaign runs across the seven agents
- refinement-loop.txt -- how a user refines output without a full re-run
- wireframes.md -- the result-page UI decisions and rationale

Each decision log in decision_logs/ points at the relevant design document and diagram. decision_logs/DL-SUMMARY.txt tells the whole build as a five-sprint story, and diagrams/00-INDEX.txt describes every diagram.

Stack

```

Layer | Tool
-----
Web framework | Flask
-----
Vision inference | OpenRouter (free tier)
-----
Creative text inference | Ollama + gemma4:e2b
-----
Structured inference | Ollama + mistral:7b
-----
Embeddings | Ollama + nomic-embed-text
-----
Vector store | ChromaDB (local, persistent)
-----
Color extraction | Pillow + ColorThief
-----
PDF parsing | pypdf
-----

```